

Intern Resume Evaluator Project Report

-Nivasini Muthukumaran

Introduction

The project implements a resume screening application that evaluates uploaded PDF resumes against a given job or internship description and presents ranked results through a Streamlit interface. The system combines PDF text extraction, keyword-based information retrieval, TF-IDF-based text similarity, and score-based candidate ranking to support faster shortlisting.

Problem Statement

Manual resume screening is time-consuming, repetitive, and difficult to scale when many candidates apply for the same role. A lightweight application that can automatically compare resumes with role requirements helps reduce initial screening effort and gives recruiters or project evaluators a faster way to identify relevant profiles.

Objectives

The primary objective of the project is to build an internship-oriented resume evaluation tool that can read multiple resumes, compare them against a job description, and rank candidates using measurable criteria. A second objective is to extract useful profile information such as contact details, degree-related keywords, technical skills, and experience indicators so that the user can review both numeric scores and supporting evidence.

Scope

The scope of the project includes PDF upload, resume text extraction, job-description matching, skill detection, experience estimation, result ranking, filtering, and report download in CSV and Excel formats. The current implementation is intended for basic screening and does not use semantic embeddings, named-entity recognition pipelines, or learned classification models.

Technologies Used

The two applications are implemented in Python and use Streamlit for the web interface, PyPDF2 for PDF text extraction, pandas for tabular result handling, regular expressions for pattern matching, and scikit-learn for TF-IDF vectorization and cosine similarity scoring. Excel export is generated with pandas and the openpyxl engine through an in-memory buffer.

System Overview

The user begins by pasting a job or internship description into a text area and uploading one or more resumes in PDF format. Each resume is processed to extract raw text, normalize the content, detect profile details, compute several matching scores, and append the result to a final ranked DataFrame that is displayed in the Streamlit dashboard.

Workflow

1. Accept a job description through the Streamlit interface.
2. Upload one or more PDF resumes.
3. Extract text from each resume using `PyPDF2.PdfReader`.
4. Preprocess the text by converting it to lowercase and removing unwanted symbols with regular expressions.
5. Extract profile fields such as email, phone number, degree information, skills, and experience years.
6. Compute text match, skill match, and experience-based score components.
7. Generate a final fit score and assign a review label such as Good Fit, Worth Reviewing, or Low Fit.
8. Display ranked results and allow CSV or Excel download.

Module Description

1. User Interface Module

The user interface is built with Streamlit and provides the project title, a text area for the job description, a multi-file PDF uploader, a button to trigger analysis, summary metrics, a filtered results table, and download buttons. Both files follow the same interface structure, which makes the application easy to demonstrate and use in a browser.

2. Resume Text Extraction Module

The `extract_resume_text` function opens each uploaded PDF and concatenates the text extracted from all pages. This module converts unstructured resumes into plain text that can be cleaned and analyzed by the later scoring functions.

3. Text Preprocessing Module

The preprocessing stage converts text to lowercase, removes non-alphanumeric symbols except selected characters, and normalizes whitespace. This step ensures that resume text and job descriptions are in a consistent format before similarity and keyword checks are applied.

4. Information Extraction Module

Both codes extract email and phone number using regular expressions and detect degree-related keywords through a list-based scan of the resume text. They also estimate experience years by looking for common textual patterns such as “2 years” or “3 yrs.”

5. Skill Matching Module

The skill matching logic identifies technical skills present in the resume and compares them with the skills implied by the job description. In the simpler implementation, this comparison uses a flat skill list, while the more advanced version uses canonical skills with aliases and separates required skills from preferred skills.

6. Similarity Scoring Module

Both programs use TF-IDF and cosine similarity to measure textual overlap between the job description and the resume. The advanced version improves this by using bigrams and English stop-word removal, which can produce a more context-aware text match score.

7. Final Evaluation Module

The final evaluation combines multiple score components into a single role-fit score that determines candidate ranking. The simpler version uses a fixed weighted average of text, skill, and experience scores, while the advanced version changes the weighting for internship roles and also includes a project score.

8. Reporting and Export Module

The application stores all candidate results in a pandas DataFrame, sorts them by final score, and displays summary metrics for the best profiles. It also supports filtered viewing and export to CSV or Excel using in-memory file generation.

Methodology

The project follows a rule-based and similarity-driven methodology rather than a trained machine-learning classification approach. Resume text is first extracted and normalized, then profile features are detected using regular expressions and keyword lists, and finally multiple scoring functions estimate candidate suitability for the given role.

For textual relevance, the system applies TF-IDF vectorization and cosine similarity to compare the resume against the job description. For skill relevance, the code checks whether the candidate possesses the technical keywords expected by the role. For profile suitability, the code estimates experience years and, in the more advanced version, also computes a project score and adapts the weighting for internship postings.

Comparison of app.py and app1.py

Overview of the two versions

Both files implement the same overall project idea and share the same front-end workflow, but they differ in the depth of matching logic and the sophistication of the scoring model. The `app.py` version is simpler and easier to explain, while `app1.py` introduces more advanced screening logic intended to improve candidate evaluation quality.

Key differences

Aspect	app.py	app1.py
Skill source	Uses a flat SKILL_BANK list for direct matching.	Uses SKILL_ALIASES to map multiple related terms to one canonical skill.
Skill extraction	Matches exact keywords in the resume text.	Matches aliases such as ml, sklearn, torch, and natural language processing to broader skill categories.
Job-skill logic	Compares detected resume skills with skills found in the job description without separating priority types.	Separates skills into required and preferred categories using marker phrases such as “must have” and “nice to have.”
Skill score	Uses a straightforward matched-versus-missing skill ratio.	Uses weighted scoring with 75% weight for required skills and 25% for preferred skills.
Text similarity	Uses standard TF-IDF with default settings.	Uses TF-IDF with ngram_range=(1, 2) and English stop-word removal.
Internship awareness	Does not explicitly treat internship roles differently.	Detects internship roles using keywords such as intern, fresher, and entry level.
Experience scoring	Applies the same experience scoring logic to all roles.	Gives internship roles a full experience score and handles them separately from experienced roles.
Project score	Not included.	Adds a Project Score based on overlap between project-related keywords in the job description and resume.
Final score formula	Uses a fixed formula: 40% text match, 40% skill match, and 20% experience match.	Uses different formulas; internship roles include skill, text, experience, and project score, while non-internship roles exclude project score.
Complexity	Easier to understand and present in a classroom setting.	More advanced and closer to a real shortlisting workflow.

Analytical discussion

`app.py` is better suited to introductory project explanation because the functions are easier to follow and the scoring design is simple and transparent. `app1.py` is technically stronger because it handles skill aliases, required-versus-preferred skills, internship-specific scoring, and project relevance, which make the evaluation process more realistic for practical screening.

The main trade-off is simplicity versus screening depth. A reviewer looking for clarity and easy implementation may prefer `app.py`, while a reviewer looking for richer logic and better adaptation to internship hiring scenarios may prefer `app1.py`.

Advantages of the Project

- Reduces time spent on manual resume screening.
- Supports upload of multiple resumes in one run.
- Provides interpretable score components instead of only one raw score.
- Extracts candidate details such as contact information, skills, and education cues.
- Generates downloadable CSV and Excel reports for later review.
- Offers a browser-based interface that is simple to use and demonstrate.

Limitations

The project relies heavily on keyword and pattern matching, so it may miss semantically similar skills that are written in unexpected ways. PDF text extraction quality also depends on the structure of the original document, and unusual resume formats may reduce extraction accuracy.

The system does not use semantic embeddings, contextual language models, or advanced entity recognition, so it cannot fully understand resume meaning beyond the supplied rules and vector overlap. Experience estimation is also approximate because it depends on explicit textual year patterns rather than date-range reasoning.

Applications

This type of application can be used for internship screening, entry-level role filtering, campus recruitment support, and small-scale recruiter assistance tools. It is also useful as an educational mini-project to demonstrate NLP preprocessing, information extraction, similarity scoring, and Streamlit-based deployment in one integrated workflow.

Future Scope

Future improvements could include semantic skill matching using transformer embeddings, stronger resume parsing with named-entity recognition, support for DOCX resumes, and better project or education analysis. The system could also be extended with recruiter-side filters, minimum-threshold rules, visualization dashboards, and database-backed candidate history tracking.

Conclusion

The Intern Resume Evaluator demonstrates how a practical screening workflow can be built using Python, Streamlit, regular expressions, and TF-IDF-based similarity scoring. Both code versions successfully implement resume upload, text extraction, profile analysis, ranking, and downloadable reports, but `app1.py` provides a more advanced scoring strategy while `app.py` offers a simpler and more presentation-friendly structure.

Overall, the two files represent two stages of the same project: **one optimized for clarity and one optimized for stronger matching logic.**